

Computing Consistent Levels for Layered Software Architecture Visualizations

Separating Class Order, Package Order and Local Layout

Johannes Weigend

Weigend AM GmbH & Co.KG, Germany/Brazil
johannes.weigend@weigend.de

Michael Philippsen

Friedrich-Alexander-Universität Erlangen-Nürnberg, Germany
michael.philippsen@fau.de

Keywords: software architecture visualization, layout invariants, layered architecture visualization, strongly connected components, dependency analysis, software engineering tools

Abstract: Layered architecture visualizations communicate dependency structure by means of vertical positioning: elements that depend on others sit higher, foundational elements sit lower. To be of practical value their upward edges should either give real architectural feedback or indicate exposed cuts. However, in many visualizations upward edges are layout artifacts rather than findings — drawings that can look plausible but mislead the viewer.

Levelized dependency maps are not new. Structure101’s Levelized Structure Map arranges elements into rows. Published descriptions specify the intended visual behaviour but not the concrete level-computation algorithm or its consistency conditions, leaving it unclear whether an upward edge is a finding, a tolerated cycle cut, or an implementation defect.

In this paper, we present the level computation pipeline implemented in S202, an open-source Java architecture-visualization tool under Apache 2.0.^a The pipeline has four phases, executed in a single forward pass. First, S202 detects global SCCs in the class graph and heuristically cuts large cycles. Second, from weighted package dependencies it derives an architectural hypothesis: more heavily used packages sit lower, their users higher. Third, it computes a local level per parent container that determines vertical placement of siblings. Fourth, a horizontal-ordering pass shortens arrows without touching levels or invariants.

The algorithmic contribution is the strict separation of architectural hypothesis from layout position. Package analysis decides which package directions are normal, cyclic, or heuristically cut; local placement only decides where classes and visible containers appear inside one concrete parent. This prevents feedback loops in which class feedback, package order, and local positions redefine each other.

Five machine-checkable invariants act as implausibility alerts to the developer: class dependencies respect class levels after heuristic cuts, dominant weighted package dependencies respect package levels, cyclic peer packages share a level after filtering, visual ranks follow local levels, and cached edge classifications match the current SCC and level state. Four of these alerts are pipeline-defect detectors that never fire on a correct run; the fifth fires only on remaining upward edges from broken cycles and points to a concrete architecture violation. On an acyclic codebase, S202 therefore produces an alert-free layout in which every edge points downward.

We demonstrate the pipeline on Minecraft Forge 1.19.2, a large and highly cyclic system that stresses every stage. Separating global architecture depth from local layout, and preserving the computed package order in the UI instead of reordering it to suppress cycles, reduces the visual upward-edge count from roughly 6000 to 4157; the remaining edges are real architecture violations or explicit heuristic cuts.

The result is a reproducible level-computation algorithm: every visible anomaly is a finding, not an artifact.

^a<https://github.com/jweigend/S202>

REFERENCES

- Dashuber, V., Philippsen, M., and Weigend, J. (2021). A layered software city for dependency visualization. In *Proceedings of the 12th International Conference on Information Visualization Theory and Applications (IVAPP)*, VISIGRAPP 2021, pages 15–26. SCITEPRESS. Best Paper Award.
- Dashuber, V., Philippsen, M., and Weigend, J. (2022). Static and dynamic dependency visualization in a layered software city. *SN Computer Science*, 3(4):305.
- Eades, P., Lin, X., and Smyth, W. F. (1993). A fast and effective heuristic for the feedback arc set problem. *Information Processing Letters*, 47(6):319–323.
- Headway Software Technologies Ltd. (2023a). Dependency matrix. Structure101 Studio documentation. Available at <https://www.sonarsource.com/structure101/docs/java/studio/content/graphs/matrix>, accessed 15 May 2026.
- Headway Software Technologies Ltd. (2023b). Levelized structure map (lsm). Structure101 Studio documentation. Available at <https://www.sonarsource.com/structure101/docs/cpa/studio/Content/restructure101/lsm.html>, accessed 15 May 2026.
- Karp, R. M. (1972). Reducibility among combinatorial problems. In Miller, R. E. and Thatcher, J. W., editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press.
- Muccini, H. and Tekinerdogan, B. (2012). Software architecture tool demonstrations. In *Joint 10th Working Conference on Software Architecture and 6th European Conference on Software Architecture, Companion Volume*, WICSA/ECSA 2012, pages 84–85. ACM.

Appendix A – Algorithm in Detail

This supplementary material expands the one-page abstract.

Bird’s-eye view

The pipeline is a single forward pass through four deterministic phases. There is no iteration, no fixed-point loop, no search for a minimum-violation arrangement, and no resorting after the fact.

- **Phase 1 — Class order.** Collapse the class dependency graph into SCCs, break large SCCs by a degree-rank heuristic, then assign every class an *architecture level* by longest path on the SCC-DAG. Feeds the tangle view, the SCC display, and the class-level consistency check; does not drive the 2D (x, y) layout, but the class architecture level can be mapped to z in 3D views as a complexity-height encoding.
- **Phase 2 — Package order.** On the weighted inter-package graph, break SCCs by a weight-rank heuristic, then assign every package an *architecture level*. Feeds the package-level consistency checks and is consumed by Phase 3 as a hard constraint.
- **Phase 3 — Place classes into the package frame.** Phase 2 has fixed the vertical order of packages; Phase 3 leaves that order intact and only places the classes within each parent. For each parent container, it builds the sibling-only graph from class-call counts, breaks any local cycles by first cutting edges that contradict the package architecture (so Phase 2’s order is never touched), and assigns each class a *local level* by longest path on the SCC-DAG. Sub-package siblings keep the order Phase 2 gave them; only classes are newly positioned.
- **Phase 4 — Horizontal arrangement.** Within each row, place siblings horizontally so inter-sibling arrows are short and rarely cross. Does not change any level and is deliberately excluded from the layout invariants.

Where the arrows come from. The arrows in the view are not produced by the four pipeline phases. They are the class-to-class dependencies extracted from the bytecode in the very first analysis step, long before any level is computed. Phases 1–3 decide *where the boxes go*; the renderer then simply draws an arrow from each source box to each target box. Every arrow falls into one of four classes, assigned by the pipeline before rendering:

- `NORMAL` – source box ends up above target box (downward arrow);
- `VIOLATION` – source box ends up below target box even though the edge was not cut (upward arrow, drawn red);
- `INTRA_SCC` – both endpoints sit in the same cyclic group (sideways arrow within a peer level);
- `HEURISTIC_CUT` – the edge was explicitly removed by the SCC-breaker to obtain a usable hierarchy (also drawn red, but as a deliberate cut, not a finding).

The five invariants then *read* the placed boxes and the classified arrows. They do not change either; they only check that the two agree with the architecture data and with each other. Concretely, R1-algo and R1-visual check the agreement between an arrow’s direction and its endpoints’ levels; R2 and R3 check the package-level placement; R5 checks that the cached arrow classification still matches the current levels.

Invariants act as implausibility alerts. The invariants run *after* the pipeline (R is short for *rule*; the catalogue contains R1–R5, with R4 retired and absorbed into R2, see Section “R4: A Retired Rule”). They are not a feedback signal that drives further iteration. Four of them (R1-algo, R2, R3, R5) are pipeline-defect alerts that never fire on a correct run; the fifth (R1-visual) fires when a remaining upward arrow points to a real architecture violation. Every visible upward arrow in the resulting drawing is therefore one of three things: an architecture violation (`VIOLATION`, raising an R1-visual alert), an explicit `HEURISTIC_CUT`, or an `INTRA_SCC` edge that could not be broken without contradicting the package architecture — never an accidental artifact of the layout algorithm.

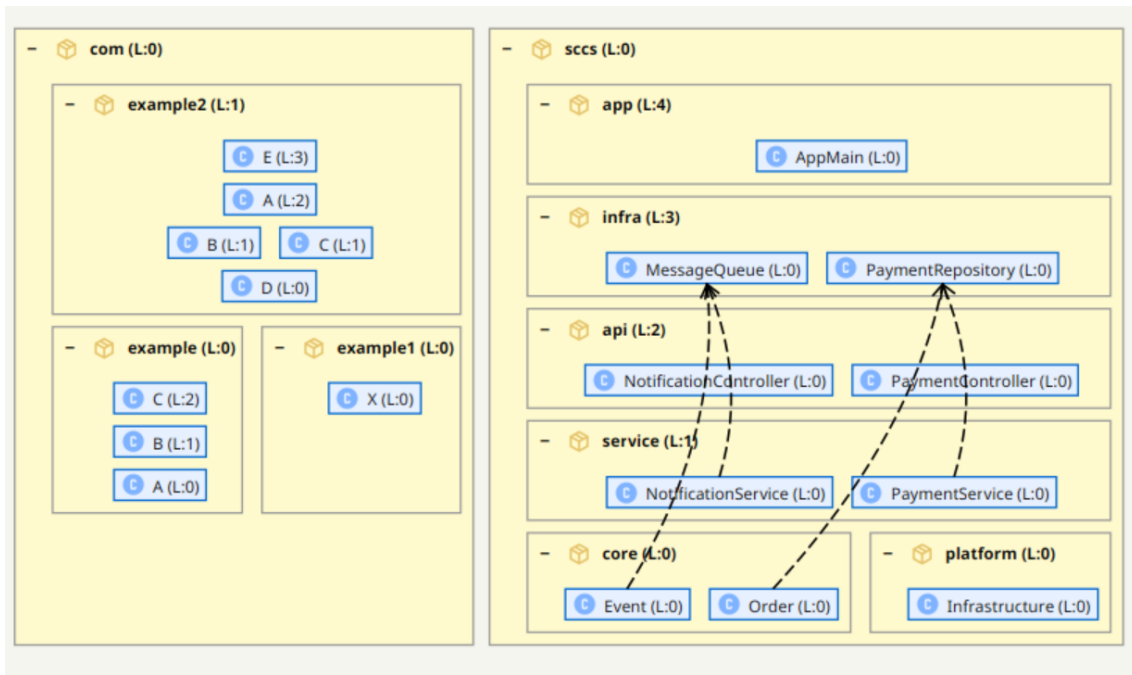


Figure 1: The S202 ArchitectureView with class and package boxes. **Left:** an acyclic situation taken from the test JAR (`com.example...`) — no upward edges, no alerts. **Right:** a real-world cycle between `core`, `infra` and `service`; the remaining upward edges trigger R1-visual alerts that flag concrete architecture violations.

Figure 1 shows the two layout regimes the pipeline has to handle. Left: an acyclic project, where every dependency points downward and the architecture reads at a glance. Right: a real-world cycle between `core`, `infra` and `service`, where the remaining upward edges are exactly the architecturally interesting findings that the R1-visual alert surfaces rather than hides.

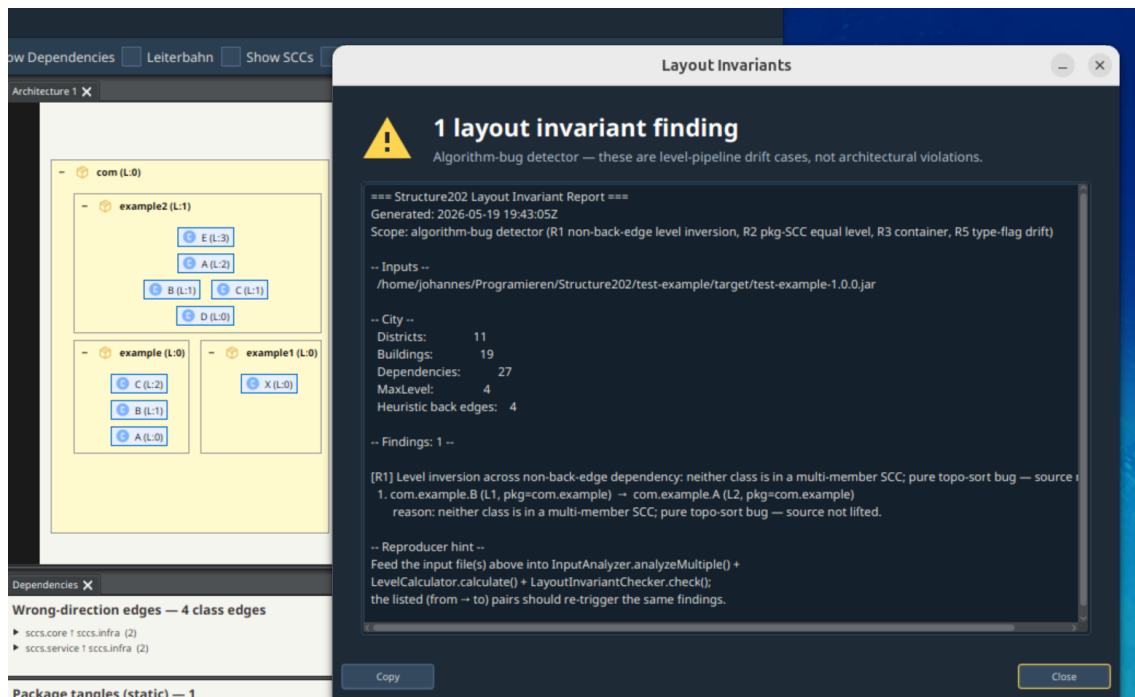


Figure 2: Implausibility alert in action. For this screenshot we deliberately injected a level-pipeline defect into LevelCalculator that misplaces class `com.example.A` on the wrong level (compare with Figure 1, left, which shows the correct placement of the same test JAR). The post-pipeline invariant checker catches the inconsistency immediately and pops up the *Layout Invariants* dialog: a single R1 finding with a copy-paste reproducer block tells the developer that the dependency `B → A` now runs against the computed levels. The status bar additionally reports the four wrong-direction class edges induced by the bug. No human inspection of the picture was needed to surface the defect; the alert is generated automatically by the consistency checks that run after every analysis.

The dialog text in Figure 2 is small at print size, so the same report is reproduced verbatim below as it appears in the log and the dialog:

```
16:43:05.166 [JavaFX Application Thread] WARN de.weigend.s202.ui.wfx.S202Module
-- Layout invariant report (1 finding(s)):
=== Structure202 Layout Invariant Report ===
Generated: 2026-05-19 19:43:05Z
Scope: algorithm-bug detector (R1 non-back-edge level inversion,
      R2 pkg-SCC equal level, R3 container, R5 type-flag drift)

-- Inputs --
/home/johannes/Programieren/Structure202/test-example/target/test-example-1.0.0.jar

-- City --
Districts: 11
Buildings: 19
Dependencies: 27
MaxLevel: 4
Heuristic back edges: 4

-- Findings: 1 --

[R1] Level inversion across non-back-edge dependency: neither class is
      in a multi-member SCC; pure topo-sort bug -- source not lifted.
1. com.example.B (L1, pkg=com.example) -> com.example.A (L2, pkg=com.example)
      reason: neither class is in a multi-member SCC;
      pure topo-sort bug -- source not lifted.
```

```
-- Reproducer hint --
Feed the input file(s) above into InputAnalyzer.analyzeMultiple() +
LevelCalculator.calculate() + LayoutInvariantChecker.check();
the listed (from -> to) pairs should re-trigger the same findings.
```

The remainder of this appendix walks through the algorithm in detail: the core problem and four failure modes that motivate the three-coordinate model, the solution, the layout invariants, the complete pipeline, the *software-ekg-7* fix that the invariant checker caught, a summary, and a Q&A section. Appendix B contains the reference implementation.

The Core Problem: Architectural Hypothesis, Local Layout, Five Consistency Checks

A correct layered layout has to separate an architectural hypothesis from the local placement that makes this hypothesis visible. In S202 this means answering three related questions that are often treated as one ordering problem:

- *Class analysis* – where does a class sit in the global class dependency stack? This is the longest-path depth in the SCC-collapsed class graph.
- *Package analysis* – what package order is implied by the aggregated dependency traffic? This architectural hypothesis is computed on the weighted inter-package graph, independently of the deepest contained class.
- *Local layout* – where should an element’s box sit inside its parent container? Only dependencies between classes in the subtrees of that parent’s direct children decide this local level.

The three computed values are independent in principle, and the implementation must keep the package-level hypothesis separate from local layout choices. Mixing them is the root cause of every failure mode below. Five consistency checks then inspect the resulting levels and rendered order:

1. **Class-level consistency (R1-algo):** If $A \rightarrow B$ denotes a class dependency, then $\text{archLvl}(A) > \text{archLvl}(B)$ after heuristic back-edges are removed. This checks the longest-path assignment in the class-dependency SCC-DAG.
2. **Visual-rank consistency (R1-visual):** For every class dependency $A \rightarrow B$ that is not a heuristic back-edge, the rendered position of A in its parent box chain sits above B . The comparison is lexicographic on the tuple of local levels along the parent path, ending with the class’s own local level. This is the statement represented by the drawing.
3. **Package-SCC consistency (R2):** Packages that form a cyclic group in the filtered package dependency graph share the same package architecture level.
4. **Package-order consistency (R3):** In the weighted inter-package dependency graph, when the call traffic from package P_A to P_B dominates the reverse, P_A ends up at a strictly higher package architecture level than P_B .
5. **Classification consistency (R5):** Cached edge classifications such as `NORMAL`, `VIOLATION`, and `INTRA_SCC` match the current level assignment and SCC membership.

R1-algo, R2, R3 and R5 check the algorithmic model; R1-visual checks the rendered order. The failure modes that follow show why each obvious shortcut collapses one of these concerns into another and loses information.

Relation to Structure101 and Levelized Structure Maps

Structure101’s Levelized Structure Maps are a closely related practical predecessor to the layout studied here. They organize code elements into rows such that, as far as possible, dependencies flow downward, and they expose feedback dependencies when cyclic regions prevent strict levelization (Headway Software Technologies Ltd., 2023b; Headway Software Technologies Ltd., 2023a). This makes Structure101 an important point of comparison and a necessary part of the related work.

The public Structure101 documentation, however, describes the intended properties of the visualization rather than the concrete algorithm that computes the rows. The same is true for academic references that discuss Structure101 as a software architecture analysis tool or as related work for layered visualization (Muccini and

Tekinerdogan, 2012): they characterize the LSM concept, but do not provide an algorithmic specification of Structure101’s levelization procedure. To the best of our knowledge, no published paper describes the exact Structure101 algorithm.

S202 therefore provides an open algorithmic account of a related class of leveled dependency layouts, not a claim that the visual idiom itself is new. The novelty lies in making the computation explicit, separating class architecture depth, package architecture order, and local layout position, and checking the resulting layout through executable invariants.

S202 also continues a line of work the authors have published on layered dependency visualization. (Dashuber et al., 2021) introduced a layered software-city metaphor that uses building height for architectural depth, and (Dashuber et al., 2022) extended it with static and dynamic dependency overlays. Both papers describe the visual idiom and the supporting infrastructure but treat the underlying level computation as a single coordinate; they do not separate class architecture depth, package architecture order, and local layout position, and they do not introduce machine-checkable invariants. The present paper closes this algorithmic gap for the 2D layered view in S202.

Failure Mode 1: Cascading Level Updates

An algorithm iterates over classes and assigns levels. It first sees class A in package P_1 , computes $\text{level}(A) = 3$, and therefore sets $\text{level}(P_1) = 3$. Later it reaches class B in package P_2 and discovers that A and B are in the same SCC and must share a level, now $\text{level}(A) = \text{level}(B) = 5$. This forces P_1 to be recomputed. If P_1 depends on P_3 , P_3 may need to move as well. A cascade follows.

In a naive single-pass implementation, however, P_1 and P_3 may already have been finalized. SCC membership is only known after the relevant graph has been traversed; if level assignment and SCC discovery are interleaved, the iteration order decides whether the correction is still applied in time.

Failure Mode 2: Interleaved Cycle Detection Produces Classpath-Dependent Results

The most subtle practical failure mode occurs when cycle detection and level computation are not separated. The algorithm traverses the class graph, assigns levels, and handles discovered cycles on the fly, depending on which class happens to be processed first.

Classes come from JARs. The order in which the JVM sees classes follows the order in which they were packaged into the JAR. That order can depend on the build tool, module order in a multi-module build, filesystem ordering on the build server, and other implementation details. None of these orders is semantically meaningful; none encodes architectural dependency direction.

The result is unreliable: the same source code can produce different level layouts under different build configurations. The errors are subtle. The layout may still look plausible, arrows may mostly point in the expected direction, and no obvious visual break appears. Yet the precise hierarchy information is corrupted and may remain unnoticed for a long time.

The solution is to separate the phases strictly: first compute the SCC partition on the complete graph, then build the SCC-DAG, then assign levels. Tarjan’s algorithm computes the SCC partition in $O(V + E)$; with deterministic input ordering, the implementation becomes reproducible and independent of classpath traversal artifacts.

Formal Counterexample: Why a Naive Recursive Algorithm Fails

The intuitive level-computation algorithm is a recursive DFS with memoization:

```
level(A) :
  if A already computed: return level(A)
  if A has no dependencies: return 0
  return max(level(dep) for dep in A.dependencies) + 1
```

For acyclic graphs this is correct and equivalent to longest-path computation on a DAG. For cyclic graphs it fails structurally.

Counterexample with two SCCs:

SCC₁ = {A, B}, with A → B → A
 SCC₂ = {C, D}, with C → D → C
 Cross dependency: A → C

The correct levels are level(C) = level(D) = 0 and level(A) = level(B) = 1.

Recursive algorithm, starting at A:

```
level(A)  [A "in progress", sentinel = 0]
-> level(B)
    -> level(A): cycle detected, return 0
    B.level = 0 + 1 = 1
-> level(C)  [C "in progress", sentinel = 0]
    -> level(D)
        -> level(C): cycle detected, return 0
        D.level = 0 + 1 = 1
    C.level = 1 + 1 = 2    <-- wrong; correct would be 0
A.level = max(1, 2) + 1 = 3    <-- wrong; correct would be 1
```

After SCC equalization, SCC₁ = max(3, 1) = 3 and SCC₂ = max(2, 1) = 2. Both results are wrong.

The cause is that the recursive algorithm enters SCC₂ from the context of SCC₁. The sentinel-based local cycle handling inflates the level of SCC₂, and that inflated level then feeds into the computation of SCC₁. The correct level of SCC₂, namely zero, is only known after the graph has been collapsed into SCCs and evaluated as an SCC-DAG.

Thus, recursive level algorithms that mix traversal, cycle handling, and level assignment can produce traversal-order-dependent results on graphs with interacting SCCs. In JVM-based tools, that traversal order can be influenced by classpath and packaging order. The correct solution collapses SCCs before level assignment: Tarjan, then SCC-DAG, then longest path.

Failure Mode 3: Deriving Package Levels from Class Levels

An obvious implementation assigns each package a level equal to the maximum level of its contained classes, then lifts parent packages transitively. This is wrong for two reasons.

First, the rule conflates containment with dependency. A package that *contains* a class at level 7 is not necessarily architecturally positioned at level 7 itself; it may have no cross-package dependencies at all. Two disconnected package trees processed from the same JAR would be ordered relative to each other by accident of their class contents, not by their actual dependency relationships.

Second, the approach creates a circular dependency between the two level systems: class levels depend on the class graph, but package levels derived from class levels then constrain the visual position of packages relative to classes, making the combined coordinate system ill-defined.

S202 avoids this by computing class levels and package levels on independent graphs with no mutual constraint. Each system starts at zero and grows according to its own dependency structure.

Failure Mode 4: Large SCCs Without Heuristics Collapse the Layout

Without a mechanism for breaking large SCCs, all members of a cyclic group must be placed on the same level. For small cycles of two to five classes this is acceptable. For Minecraft Forge 1.19.2 (5 123 classes), however, we observed a single SCC containing 2 857 classes. Without a breaker, more than 55 percent of the project would be assigned the same level. All remaining classes would be distributed above, but the internal dependency hierarchy within this large cycle would be invisible.

The heuristic SCC breaker solves this practical problem. It identifies back edges—dependencies that run against the inferred architecture direction—and removes them for level computation. The removed edges are visualized as violations instead of being ignored. The layout preserves visibility of cyclic dependencies while still producing a usable hierarchy.

Relation to Feedback Arc Set Heuristics

Failure Mode 4 is an instance of the Minimum Feedback Arc Set problem: remove a minimal set of directed edges so the graph becomes acyclic. The problem is NP-hard (Karp, 1972), so any practical polynomial-time implementation must use a heuristic or approximation.

A well-known related heuristic is due to Eades, Lin, and Smyth (Eades et al., 1993). It orders nodes according to the difference between outgoing and incoming degree, then classifies edges that point backward in the resulting sequence as feedback edges. The intuition is similar to S202: high out-degree suggests a higher-level element; high in-degree suggests a lower-level element.

S202 uses a deliberately simpler variant: a direct normalized score per node,

$$r(P) = \frac{\text{outWeight}(P) - \text{inWeight}(P)}{\max(1, \text{outWeight}(P) + \text{inWeight}(P))},$$

where the weights are summed within the SCC under consideration. At the class level the weights are the in- and out-degrees in the class dependency graph; at the package level they are aggregated method-call counts. An edge $P_A \rightarrow P_B$ inside an SCC is classified as a back-edge candidate when $r(P_A) < r(P_B) - 0.1$. Equal-rank pairs represent symmetric bidirectional dependencies with no dominant architectural direction; they remain in the same SCC and are assigned the same level.

The important distinction is transparency. Many tools break cycles implicitly, but S202 classifies heuristic cut edges explicitly. They are marked as `VIOLATION`, rendered in red, and known to the invariant checker. This makes algorithmic defects separable from tolerated heuristic artifacts.

The Solution: Three Computed Values on Independent Graphs

Failure modes 1 to 3 share a root cause: conflating values that belong to different semantic domains. S202 resolves this by computing three independent values, two of them for analysis and one for layout.

- **Class architecture level** is the longest path in the SCC-collapsed class dependency DAG. It represents global architectural depth: how many layers of dependencies separate a class from the bottom of the analysed dependency graph.
- **Package architecture level** is the longest path in the SCC-collapsed weighted inter-package dependency graph. Edge weights are method-call counts aggregated from class-level references. The result represents the module's position in the system's overall dependency order, independently of the class-level coordinate.
- **Local level** is the longest path in the SCC-collapsed sibling-only dependency graph of a single parent container. The graph contains only edges whose source and target both lie in some direct child of that parent; references leaving the parent's subtree are excluded because they are handled by the enclosing container. Every element (class and package) gets a local level that is only meaningful within its own parent box. The renderer sorts each parent's children by this index, so the visible Y position of a box is the local level, not the global architecture level.

All three concepts use the same algorithmic shape—Tarjan SCC detection, weight-based back-edge identification, DAG condensation, longest-path propagation—but on three different graphs with three different scopes. Because the systems are independent, packages with no cross-package dependencies correctly land at architecture level 0 regardless of the class levels inside them, and a top-level class like *ArchitectureView* that orchestrates a sub-package such as *ui.rendering* sits visually above the sub-package box without distorting either of their architecture levels.

The separation is an engineering choice driven by semantic clarity. A unified single-coordinate approach is theoretically possible, but it conflates three semantically distinct concerns—global architectural depth, module dependency order, and per-container visual position—into one number, making the pipeline hard to reason about, test, and maintain. Each simplification of such a unified approach turns into one of the failure modes above.

The three concepts have the following algorithmic structure. Phases 1 and 2 compute the global architecture levels; Phase 3 derives the local level per parent container from the architecture-level output.

Algorithm 1: Phase 1 – Global Class Levels (LevelCalculator, Step 3)

class dependency graph
→ Tarjan (SCCs)
→ SCCBreaker: identify heuristic back-edges in large SCCs ($|S| \geq 3$; 2-class SCCs are equalized instead)
 $\text{rank}(P) = (\text{outDeg} - \text{inDeg}) / \max(1, \text{outDeg} + \text{inDeg})$
 edge $A \rightarrow B$ is a back-edge candidate if $\text{rank}(A) < \text{rank}(B) - 0.1$
→ Remove back-edges → filtered class DAG
→ Condense into SCC-DAG → longest-path pass
→ Assign SCC level to all member classes (equalization implicit)

Algorithm 2: Phase 2 – Package Architecture Levels (LevelCalculator, Step 4)

Build weighted inter-package dependency graph $G_P = (V_P, E_P, w)$:
For each class C in package P_{leaf} with method calls to class D in P_{target} ($P_{\text{leaf}} \neq P_{\text{target}}$):
 $w(P_{\text{ancestor}}, P_{\text{target}}) += \text{callCount}(C \rightarrow D)$
 for every ancestor P_{ancestor} of P_{leaf} with $P_{\text{ancestor}} \neq P_{\text{target}}$
Fallback weight = 1 for structural references with no method-call data
No directional filtering: both parent→child and child→ancestor edges enter G_P . The SCC breaker decides direction by weight.

Iterative SCC-breaking (repeat until stable):
Tarjan on current graph → find SCCs with $|S| \geq 2$ (2-package SCCs broken if rank differs; equalized otherwise)

For each member $P \in S$:
 $r(P) = (\sum_{Q \in S} w(P, Q) - \sum_{Q \in S} w(Q, P)) / \max(1, \sum_{Q \in S} w(P, Q) + \sum_{Q \in S} w(Q, P))$
Edge $P_A \rightarrow P_B$ inside S is a back-edge if $r(P_A) < r(P_B) - 0.1$; remove it
Equal-rank pairs remain in the SCC → assigned the same level

Level assignment:
Condense remaining graph into SCC-DAG
Longest-path propagation on SCC-DAG
Assign SCC level to all member packages

Phase 1 produces classpath-independent class architecture levels because SCCs are collapsed before levels are assigned. Phase 2 produces package architecture levels directly from package dependencies: every class reference contributes to the weighted graph regardless of the containment relation between source and target package, and a per-SCC rank score decides which direction of a cyclic group dominates. The two architecture coordinate systems do not constrain each other—a package with no outgoing cross-package calls lands at level 0 regardless of the class levels inside it, and conversely the class levels of a package’s contents do not lift the package itself.

Algorithm 3: Phase 3 – Local Level per Parent Container (LocalLevelCalculator, Step 6)

for each parent package P with at least two direct children:
 Let $S(P)$ be the direct children (classes and sub-packages).
 Build sibling-only weighted graph G_P^{loc} on $S(P)$:
 edge $X \rightarrow Y$ with weight $\sum \text{callCount}(c \rightarrow d)$
 over all class refs $c \rightarrow d$ where c lies in the subtree of sibling X
 and d lies in the subtree of sibling Y ;
 refs leaving the parent’s subtree are excluded.
 Tarjan on $G_P^{\text{loc}} \rightarrow \text{SCCs}$.
 Within each SCC of size ≥ 2 : evaluate every candidate edge cut.
 First cut an edge that contradicts the package architecture; only then
 prefer the cut that maximally decomposes the SCC; edge weight and rank as further tie-breakers.
 Condense to SCC-DAG, longest-path pass, assign **localLevel** per sibling.

Phase 3 is a pure layout phase. It never modifies architecture levels. Each parent container gets its own

computation: a class’s local level inside *ui* carries no relationship to a class’s local level inside *ui.rendering*. The renderer sorts each parent’s children by their local level, descending. The enclosing parent’s local level handles cross-container ordering recursively. The local SCC breaker resolves local cycles—for example, two sibling sub-packages with mutual class references—silently. It removes one edge at a time, always preferring an edge that contradicts the package architecture: Phase 3 may never reorder packages relative to the order computed in Phase 2. Only when no architecture-contradicting edge is available does the breaker fall back on cycle-decomposition score, edge weight, and rank. S202 does not surface these local cycles as a separate Tangle category because the user-visible tangle list remains a global architecture statement.

After Phase 3 the level computation pipeline is finished. A fourth phase, the horizontal-arrangement pass (`HorizontalRowLayoutOptimizer`), runs purely for the legibility of inter-sibling dependency arrows: it reorders same-row siblings so related elements sit horizontally close. It modifies no level, no SCC membership, and no invariant result, and is therefore mentioned here only for completeness; the detailed algorithm appears later in this appendix.

	Phase 1 – Class	Phase 2 – Package	Phase 3 – Local Level
Graph	Class dependency	Weighted inter-package	Sibling-only weighted (per parent)
Edge weight	Unweighted	Method-call counts	Method-call counts
Scope	Global	Global	Per parent container
Meaning	Class arch. depth	Module dep. position	Visual Y in parent box
Zero point	Class with no deps	Package with no outgoing calls	Bottom sibling in box
Cycles	SCCBreaker (degree)	Weight-rank, all edges	Max-cycle-break within siblings
Used by	R1-algo, Tangles	R3, R2	R1-visual, renderer sort

Table 1: The three independent values computed by S202.

Layout Invariants for Level Correctness

The three level phases are guarded by five machine-checkable invariants that act as the implausibility alerts introduced in the abstract. All five run after the pipeline as post-checks on the fully computed model and the rendered layout; none feeds back into level assignment. In the current code base R1-algo, R2, R3, and R5 are evaluated by the `LayoutInvariantChecker`, while R1-visual is checked by the architecture builder as part of its violation-detection pass over the visual-rank tuple — a historical split with no semantic consequence; both halves act as identical post-checks. These checks are evidence that the computed model is internally consistent; they are not the primary algorithmic contribution.

Rule	Consistency check
R1-algo	Non-heuristic class dependencies respect class architecture levels: $A \rightarrow B \Rightarrow \text{archLvl}(A) > \text{archLvl}(B)$. Tests the longest-path assignment of Phase 1.
R1-visual	For every non-heuristic class dependency $A \rightarrow B$, the rendered position of A sits above B . The check compares positions lexicographically on $[L(\text{ancestor}_1), \dots, L(\text{ancestor}_n), L(A)]$ versus $[L(\text{ancestor}'_1), \dots, L(\text{ancestor}'_m), L(B)]$ where L denotes the local level. Covers same-parent, same-grandparent, and cross-tree comparisons uniformly.
R2	Packages forming a cyclic group in the filtered package dependency graph share the same package architecture level.
R3	For every weighted package edge (P_A, P_B) with $w(P_A, P_B) > w(P_B, P_A)$ that is not a back-edge cut by the SCC breaker: $\text{archLvl}(P_A) > \text{archLvl}(P_B)$.
R5	Cached edge classifications (NORMAL, VIOLATION, INTRA_SCC) remain consistent with the current level assignment and SCC membership.

Table 2: Five layout consistency checks verified after every analysis run. R1-algo, R3 and R2 check the algorithm; R1-visual checks the rendered picture; R5 keeps the cached annotations consistent.

Throughout the iterative development of the level computation pipeline, R2 surfaced implementation bugs after algorithm changes that the unit-test suite had not detected. Figure 3 shows a representative R2 report. In *software-ekg-7*, for instance, an R2 finding exposed a missing package-SCC equalization step and directly triggered a pipeline fix.

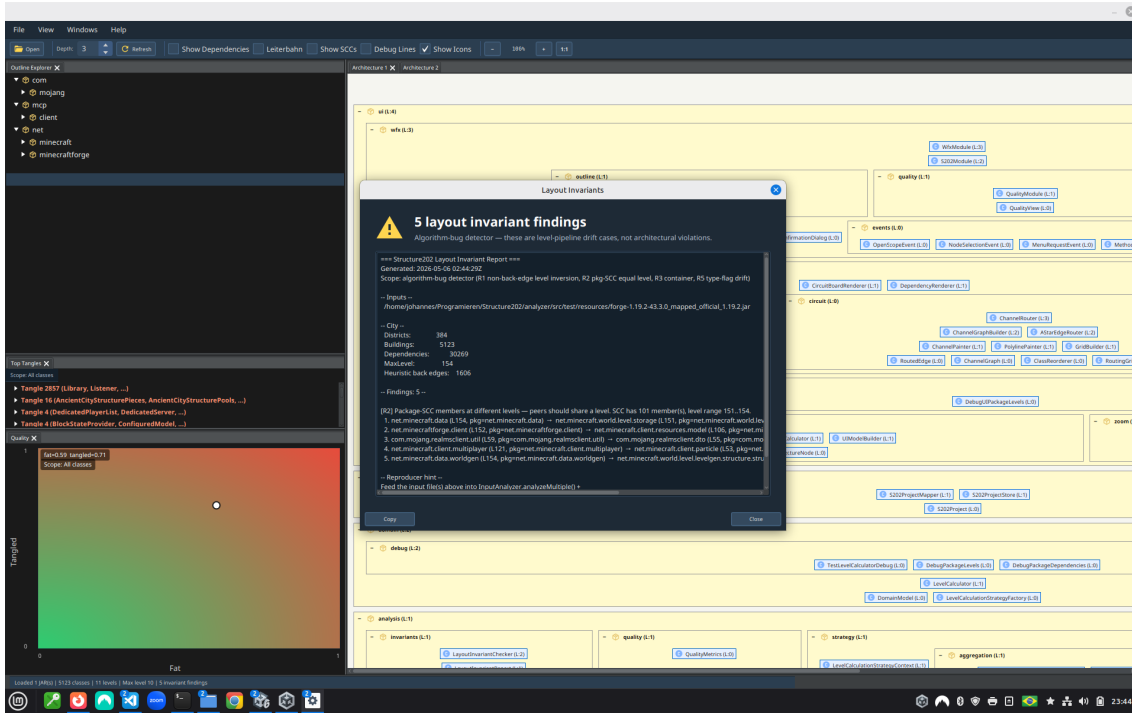


Figure 3: R2 checker report: packages in a filtered package-SCC occupy different levels. The checker reports findings as algorithm-bug candidates with a reproducer block, which makes the visual failure traceable to concrete level-pipeline consistency conditions.

R2 also guided refinement of the rule itself. Against Minecraft Forge (2 895 classes, 220 packages, 188 of 207 packages in a single package-level SCC), two R2 reports turned out to be *false positives*: R2 filtered class-level heuristic back-edges but not the package-level back-edges that the package SCC-breaker had already removed. R3 already applied the correct filter; once R2 was aligned, the Minecraft findings disappeared and all 155 invariant tests pass. The invariant checker connects the visual symptom to a reproducible algorithm defect.

The Remaining Rules: R1-algo, R1-visual, R3, and R5

R1-algo ($A \rightarrow B \Rightarrow \text{archLvl}(A) > \text{archLvl}(B)$ on the filtered class graph) is the most direct invariant on the analysis side: it is a corollary of longest-path computation on the SCC-DAG and holds trivially for any correctly implemented class-level assignment. Its value lies in providing an immediate check—a failing R1-algo signals a fundamental breakdown in the class-level pipeline.

R1-visual (lexicographic comparison of local-level tuples along the parent chain) is the invariant that the drawing represents. We separate the two checks intentionally: R1-algo asks “did Phase 1 compute consistent class architecture levels?”, R1-visual asks “does the rendered Y position reflect the class dependencies?” Both pass on a correct implementation; only the second one may fail when the sibling-only graph of some parent exposes a genuine upward edge, because that is a real architecture violation rather than an algorithm bug. The single visual-rank comparison covers every pair: siblings inside the same container compare on their final tuple element; cross-tree pairs diverge earlier; classes versus sub-package boxes in the same parent compare on the shared parent context. The comparison needs no special-case logic for parent→child versus sibling versus cross-package class references.

R3 verifies that the dominant direction in the weighted package dependency graph appears in the architecture level assignment. For any package edge (P_A, P_B) where $w(P_A, P_B) > w(P_B, P_A)$ —meaning P_A calls into P_B significantly more than vice versa—the invariant requires $\text{archLvl}(P_A) > \text{archLvl}(P_B)$. R3 excludes package back-edges identified during SCC-breaking from this check, analogously to how R1-algo excludes class-level heuristic back-edges. A failing R3 indicates that the weight-based SCC-breaking cut the wrong package edge, or that the package-level DAG propagation did not converge correctly.

R5 validates that every edge's stored classification (`NORMAL`, `VIOLATION`, `INTRA_SCC`) is still consistent with the current level assignment and SCC membership. The analysis computes edge labels and caches them in the model; after any algorithmic change they can silently become stale. **R5** is therefore a cross-cutting consistency check: it ties the structural output of the level computation pipeline back to the semantic annotations visible to the user.

R4: A Retired Rule and What It Teaches

An early version of the invariant checker contained **R4**:

```
Cyclic child packages must share a level.
```

The rule sounds reasonable and is correct in simple cases. In practice it failed because the raw package graph contains relationships that should not be interpreted as peer-level architectural cycles:

- **Parent-child edges** create apparent bidirectional relationships because a parent contains its children and children belong to their parent. Without filtering, many parent/child pairs look cyclic.
- **SCCBreaker back edges** connect packages that are not necessarily architectural peers. A rule that includes these edges equalizes the wrong candidates.
- **Shared class SCCs** can make two packages appear package-dependent without representing a true structural peer relationship.

R4 was replaced by **R2**, which expresses the same intention on the correctly filtered graph. **R2** removes class-level heuristic back-edges, package-level SCC-breaking back-edges, intra-subtree edges, and edges caused by shared class SCCs before running Tarjan on the remaining package graph.

Specifying invariants for a three-value level computation pipeline turns out to be harder than it appears. The checker therefore provides a second safety net: it not only tests the implementation against the rules, but also exposes when the rules themselves are too broad or too coarse. False positives force the rule to become more precise. The later **R2** finding in *software-ekg-7* shows that the refined rule was precise enough to distinguish real pipeline bugs from apparent ones.

Implementation: The Complete Pipeline

The conceptual structure expands into the following concrete implementation steps, which constitute the reference implementation (Appendix B).

Algorithm 1: Complete Level Pipeline (LevelCalculator)

- Step 1:** Create class objects (all levels $\leftarrow 0$)
- Step 2:** Create package objects (all levels $\leftarrow 0$)
- Step 3:** Compute class architecture levels (Phase 1)
- Tarjan on class dependency graph
 - SCCBreaker: identify heuristic back-edges in large SCCs ($|S| \geq 3$):
 - $r(C) = (\text{outDeg} - \text{inDeg}) / \max(1, \text{outDeg} + \text{inDeg})$;
 - edge $A \rightarrow B$ is a back-edge candidate if $r(A) < r(B) - 0.1$
 - Remove identified back-edges → filtered class DAG
 - Condense into SCC super-nodes
 - Single longest-path pass on condensed DAG
 - Assign SCC level to all member classes
- Step 4:** Compute package architecture levels (Phase 2)
- 4a.** Build weighted inter-package dependency graph:
 - $w(P_A, P_B) = \sum \text{method-call counts from subtree}(P_A) \text{ to } P_B$;
 - propagate weights up to all ancestor packages (no directional filtering);
 - both parent→child and child→ancestor edges enter the graph
 - 4b.** Iterative package SCC-breaking (repeat until stable):
 - Tarjan on current graph → SCCs with $|S| \geq 2$;
 - $r(P) = (\sum_{Q \in S} w(P, Q) - \sum_{Q \in S} w(Q, P)) / \max(1, \dots)$;
 - edge $P_A \rightarrow P_B$ is a back-edge if $r(P_A) < r(P_B) - 0.1$; remove it;
 - equal-rank pairs remain in the same SCC → assigned the same level
 - 4c.** Level assignment on condensed package SCC-DAG:
 - longest-path propagation; assign SCC level to all member packages.
 - No child–parent lift: layout positioning is based on `localLevel`.
- Step 5:** Set reverse dependencies (dependents)
- Step 6:** Compute local level per parent (Phase 3)
- for each** parent package P with two or more direct children:
 - build sibling-only weighted graph G_P^{loc} from class refs
 - whose source and target both live in some direct child of P ;
 - Tarjan → architecture-respecting SCC break → longest-path;
 - assign `localLevel` to each direct child of P .
-

After Step 3, a class’s architecture level is the level of the SCC node to which it belongs, measured as the longest path in the cycle-free quotient graph across all classes of the analysed project. Cycles do not influence level assignment internally because they have been collapsed before longest-path computation begins.

Step 4 treats class architecture levels and package architecture levels as independent coordinate systems. A package with no outgoing cross-package calls receives architecture level 0, regardless of the class levels inside it; two package trees with no shared dependencies are each levelled from zero in their own connected component of the weighted package graph. The earlier child–parent lift, which inflated parent package levels by the class levels their children called, is gone – it was a workaround for the missing layout coordinate.

Step 6 introduces the layout coordinate. Each parent container is treated as a self-contained scope: only class references that stay within the parent’s subtree contribute to the local sibling graph. The result is that the box for *ArchitectureView* in the *ui* package sorts above the box for the *ui.rendering* sub-package, simply because *ArchitectureView* has heavier outgoing call traffic into rendering than rendering has back into *ui*. None of this disturbs Phase 1 or Phase 2.

Algorithm 2: Phase 4 – Horizontal Row Ordering (HorizontalRowLayoutOptimizer)

for each package P :

Step 1: Group direct children by their `localLevel` value

Step 2: Collect subtree classes per row element

Step 3: Build weighted undirected proximity links from inter-sibling class dependencies

Step 4: Run bounded barycentric sweeps across rows

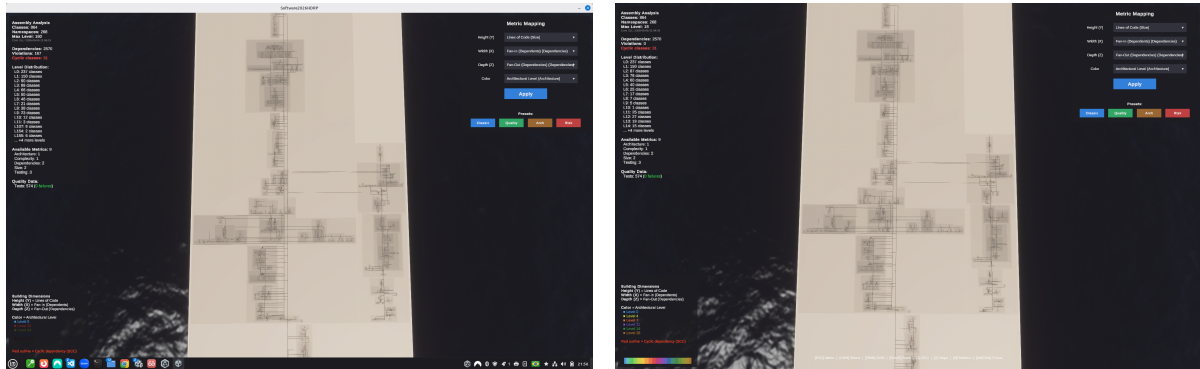
Step 5: Store the result in `horizontalLayoutOrder`

rendering: sort a copied child view by `localLevel` (Phase 3), then `horizontalLayoutOrder` (Phase 4), then full name

Phase 4 does not change any level assignment and is deliberately excluded from the layout invariants. Its only purpose is cosmetic: when an overlay of dependency arrows is displayed, same-row elements that share many inter-sibling calls are placed horizontally near each other so the arrows are shorter and less crossed. The implementation stores this as a separate `horizontalLayoutOrder` field and never mutates the semantic child list.

R2 Fix in *software-ekg-7*: Visual Evidence

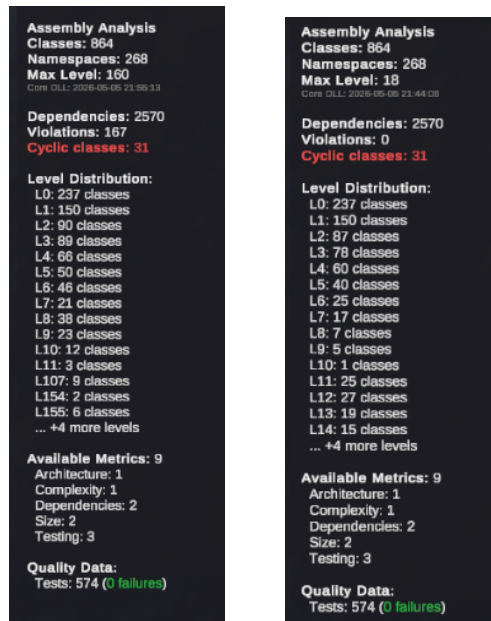
The *software-ekg-7* case is the concrete failure that motivated the package-level pipeline. Before the fix, the visualization still looked plausible: the package rectangles and dependency edges formed a readable layout, and no single visual element made the pipeline defect obvious. The metrics panel, however, exposed the inconsistency: 167 dependency violations were reported and the maximum architectural level had inflated to 160. After the package-level equalization was corrected, the same input produced zero violations and a maximum level of 18. The number of classes, packages, dependencies, cyclic classes, and tests stayed constant; the change is therefore attributable to level propagation, not to a different input model.



(a) Before the fix: the full visualization looks plausible, but the metric panel reports 167 violations and MaxLevel 160.

(b) After the fix: the same analyzed system reports zero violations and MaxLevel 18.

Figure 4: Full-screen before/after screenshots of the *software-ekg-7* visualization. The visible structure changes only subtly; the level metrics reveal the correction.



(a) Before: 167 violations, MaxLevel 160.

(b) After: zero violations, MaxLevel 18.

Figure 5: Cropped metric panels from Figure 4. These panels are the clearest user-visible symptom of the propagation defect and its fix.

Summary

Correct level computation for layered architecture visualizations requires keeping three questions separate: where does a class sit in the global class dependency stack, where does a package sit in the weighted package dependency order, and where should an element’s box go inside its parent container? Every straightforward single-pass approach over a unified graph collapses these questions into one coordinate, and the failure modes documented above show why each obvious simplification breaks down in practice.

S202 resolves this by computing three values on three independent graphs. *Class architecture levels* are the longest path in the SCC-collapsed class dependency DAG; they represent global architectural depth. *Package architecture levels* derive from a weighted inter-package dependency graph whose edges carry aggregated method-call counts; they represent module dependency position independently of class levels. *Local levels* come from the per-parent sibling-only graph closed within that parent’s subtree; they decide visual Y position inside each box without disturbing either architecture coordinate. All three use the same algorithmic shape: Tarjan SCC detection, weight-based back-edge identification, DAG condensation, longest-path propagation.

A horizontal row-ordering pass (Phase 4) improves the legibility of dependency overlays without modifying any computed value or affecting any layout invariant. Five machine-checkable invariants (R1-algo, R1-visual, R2, R3, R5) run after each analysis as implausibility alerts to the developer and emit reproducer blocks for each finding. Four of them are pipeline-defect alerts that never fire on a correct run; R1-visual fires only on remaining upward edges that point to real architecture violations. They provide regression protection and diagnostic evidence; the algorithmic contribution is the separation of the three values themselves.

Two cases demonstrate the value of the separation. In *software-ekg-7*, a plausible layered layout reported 167 dependency violations and a maximum architectural level of 160 before a package-level equalization fix; after the fix, the same input produced zero violations and a maximum level of 18 with no change to the analysed source code. On Minecraft Forge 1.19.2, introducing the local level as a distinct layout coordinate, lifting the parent filter from the weighted package graph, and preserving the computed package order in the UI instead of reordering it to suppress cycles, reduced the visual upward-edge count from roughly 6 000 to 4 157; the eliminated edges were artifacts of forcing parent packages below their sub-packages to keep them out of the algorithm’s tangle list, and the 4 157 remaining edges are real architecture violations or explicit heuristic cuts that deserve a code-level fix.

Questions and Answers

Why does S202 need a separate hierarchy for packages?

Because S202 orders classes and packages by different criteria. A class level is the depth of a node in the class dependency DAG after SCC handling: if class A depends on class B , then A belongs above B unless the edge is an explicit heuristic back-edge or both classes remain in the same SCC.

A package level answers a different question. Packages are containers, and a single package can contain many classes with dependencies in both directions. S202 therefore cannot derive package order from the deepest class inside the package. Instead, it compares the weighted relations between packages: when the traffic from package P_A to package P_B dominates the reverse traffic, P_A becomes the higher dependent package and P_B the lower package it depends on. S202 keeps equal or near-equal bidirectional traffic as a cyclic peer relationship.

This is why S202 computes class architecture levels and package architecture levels on separate graphs. The separation prevents class depth from accidentally lifting containers and prevents package aggregation from changing the meaning of class-level dependency depth.

It also prevents a feedback loop. If S202 derived package levels from class levels and then fed them back into class or layout ordering, each update could change the input of the next update. A package moves because one contained class moves; that package movement then changes sibling order or parent order; the changed order can expose another cycle or another dominant relation. Sorting and leveling algorithms then no longer operate on a stable graph. The result can depend on traversal order, classpath order, or on how often the loop is run. The system may converge accidentally, oscillate, or settle on a layout that is only an artifact of the update sequence. Separating the graphs makes each phase operate on fixed input and gives the invariants a well-defined result to check.

Why does S202 need both an architecture level and a local UI level?

Because these two numbers answer different questions. The architecture level of a class says how deep the class is in the global dependency graph: how many dependency steps separate it from the lower end of the class DAG after SCC handling. This is a system-wide analysis value.

The local level answers a layout question: where should this class or sub-package be placed relative to the other direct children of the same package? That question cannot be answered reliably from the global architecture level alone. A class can have a high global architecture level because it depends on deep chains elsewhere in the system, even though inside its own package it is not the highest local element. Conversely, a package-level coordinator may need to sit above a sub-package it calls into, even if their global architecture levels do not express that local relationship.

Using only the global DAG level for layout therefore over-constrains the picture. Elements can be pulled too high by dependencies outside their local container, and subsequent sorting passes try to repair a layout decision with the wrong input value. That is hard to make stable because the global level and the local sibling order are coupled through unrelated dependencies.

S202 keeps both values. The architecture level remains the semantic dependency depth used for global checks and reports. The local level is recomputed inside each package from the sibling-only graph and is used for visual ordering inside that package. The renderer can then place elements correctly within a container without changing what their global architecture level means.

Appendix B – LevelCalculator Reference Implementation

The following source excerpt is the complete Java implementation of the level pipeline discussed above.

```
1  /*
2   * Copyright 2026 Weigend AM GmbH & Co.KG
3   *
4   * Licensed under the Apache License, Version 2.0 (the "License");
5   * you may not use this file except in compliance with the License.
6   * You may obtain a copy of the License at
7   *
8   *     http://www.apache.org/licenses/LICENSE-2.0
9   *
10  * Unless required by applicable law or agreed to in writing, software
11  * distributed under the License is distributed on an "AS IS" BASIS,
12  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
13  * See the License for the specific language governing permissions and
14  * limitations under the License.
15  */
16 package de.weigend.s202.domain.architecture;
17
18 import de.weigend.s202.domain.DomainModel;
19 import de.weigend.s202.domain.strategy.LevelCalculationStrategyContext;
20 import de.weigend.s202.graph.StronglyConnectedComponent;
21 import de.weigend.s202.graph.TarjanSCCFinder;
22 import de.weigend.s202.reader.DependencyModel;
23
24 import java.util.*;
25 import java.util.logging.Logger;
26
27 /**
28  * Calculates architectural levels for classes and packages based on dependencies.
29  *
30  * Pipeline:
31  * Step 1 Create class objects (all levels = 0)
32  * Step 2 Create package objects (all levels = 0)
33  * Step 3 Compute class levels strategy (Tarjan → SCC-break → DAG → longest-path)
34  * Step 4 Compute package levels weighted inter-package graph → SCC-break → DAG → longest-path
35  * Step 5 Set reverse dependencies
36  * Step 6 Assign local layer index per-parent sibling graph → SCC-break → DAG → longest-path
37  * (rendering position within each parent box, no global meaning)
38  *
39  * Package levels are computed independently from class levels using a weighted
40  * inter-package dependency graph. The weight of an edge  $P_A \rightarrow P_B$  is the
41  * aggregated method-call count from classes in  $P_A$ 's subtree to classes in  $P_B$ ,
42  * with a fallback weight of 1 for structural references without method-call data.
43  * This separates the containment hierarchy from the dependency hierarchy and
44  * correctly handles disconnected package trees (they are levelled independently,
45  * both starting at 0).
46  *
47  * Package SCC-breaking uses a weight-based rank score:
48  *  $\text{rank}(P) = (\sum \text{outgoing weights within SCC} - \sum \text{incoming weights within SCC})$ 
49  *  $\quad \quad \quad / \max(1, \text{sum of both})$ 
50  * Edge  $P_A \rightarrow P_B$  is a back-edge when  $\text{rank}(P_A) < \text{rank}(P_B) - 0.1$ .
51  * Equal-rank pairs (symmetric dependency) are left in the same SCC and assigned
52  * the same level -- they are genuine peers with no dominant dependency direction.
53  */
54 public class LevelCalculator {
55
56     private static final Logger LOG = Logger.getLogger(LevelCalculator.class.getName());
57     private static final double RANK_THRESHOLD = 0.1;
58
59     private final LevelCalculationStrategyContext strategyContext;
```

```

60
61 public LevelCalculator() {
62     this(LevelCalculationStrategyFactory.createDefault());
63 }
64
65 public LevelCalculator(LevelCalculationStrategyContext strategyContext) {
66     Objects.requireNonNull(strategyContext, "strategyContext cannot be null");
67     this.strategyContext = strategyContext;
68 }
69
70 public DomainModel calculate(DependencyModel rawModel) {
71     DomainModel model = new DomainModel();
72
73     // Step 1: Create class objects (all levels = 0)
74     for (String className : rawModel.getAllClassNames()) {
75         DependencyModel.ClassInfo rawClass = rawModel.getClass(className);
76         model.addClass(className, new DomainModel.CalculatedElementInfo(
77             className, rawClass.simpleName, "CLASS", 0,
78             new HashSet<>(rawClass.dependencies), rawClass.interfaceType));
79     }
80
81     // Step 2: Create package objects (all levels = 0)
82     for (String packageName : rawModel.getAllPackageNames()) {
83         DependencyModel.PackageInfo rawPkg = rawModel.getPackage(packageName);
84         model.addPackage(packageName, new DomainModel.CalculatedElementInfo(
85             packageName, rawPkg.simpleName, "PACKAGE", 0, new HashSet<>()));
86     }
87
88     // Step 3: Compute class levels
89     calculateClassLevels(model);
90
91     // ---- DEMO_ALERT_INJECTION (review demo for M. Philippsen, 2026-05-19) ----
92     // When system property s202.demo.injectAlert=true, deliberately corrupt
93     // the architecture level of com.example.A so R1-algo fires when
94     // test-example is loaded. The local layout (which uses localLevel, not
95     // architectureLevel) is unaffected, so the visualization still shows
96     // C above B above A as expected -- but the implausibility alert dialog
97     // opens and reports the inconsistency. Tests run without the property
98     // and stay green. Remove this block once the demo session is over.
99     if (Boolean.getBoolean("s202.demo.injectAlert")) {
100         DomainModel.CalculatedElementInfo demoA = model.getClass("com.example.A");
101         DomainModel.CalculatedElementInfo demoC = model.getClass("com.example.C");
102         if (demoA != null && demoC != null) {
103             demoA.setArchitectureLevel(demoC.architectureLevel);
104         }
105     }
106     // ---- END DEMO_ALERT_INJECTION ----
107
108     // Step 4: Compute package levels from weighted inter-package graph.
109     calculatePackageLevels(model, rawModel);
110
111     // Step 5: Set reverse dependencies
112     updateDependentRelationships(model);
113
114     // Step 6: Assign per-parent local layer index -- independent of the
115     // global architectureLevel, used by the renderer to position
116     // siblings within each parent's box.
117     new LocalLevelCalculator().assign(model, rawModel);
118
119     return model;
120 }
121

```

```

122 public LevelCalculationStrategyContext getStrategyContext() {
123     return strategyContext;
124 }
125
126 // -----
127 // Step 3 -- class levels
128 // -----
129
130 private void calculateClassLevels(DomainModel model) {
131     Map<String, Set<String>> classDeps = new HashMap<>();
132     for (DomainModel.CalculatedElementInfo c : model.getAllClasses().values()) {
133         classDeps.put(c.fullName, new HashSet<>(c.dependencies));
134     }
135     Map<String, Integer> levels =
136         strategyContext.getClassLevelStrategy().calculateClassLevels(classDeps);
137     for (Map.Entry<String, Integer> e : levels.entrySet()) {
138         DomainModel.CalculatedElementInfo info = model.getClass(e.getKey());
139         if (info != null) info.setArchitectureLevel(e.getValue());
140     }
141 }
142
143 // -----
144 // Step 4 -- package levels via weighted inter-package graph
145 // -----
146
147 private void calculatePackageLevels(DomainModel model, DependencyModel rawModel) {
148     if (model.getAllPackages().isEmpty()) return;
149     Set<String> allPkgNames = model.getAllPackages().keySet();
150
151     // Build weighted graph: weight[from][to] = aggregated method-call
152     // count from classes in 'from' to classes in 'to', with fallback
153     // weight 1 for structural dependencies without method-call data.
154     Map<String, Map<String, Integer>> weights = buildWeightedPackageGraph(model, rawModel);
155
156     // Unweighted adjacency for Tarjan (direction matters, zero-weight edges excluded)
157     Map<String, Set<String>> graph = new LinkedHashMap<>();
158     for (String pkg : allPkgNames) graph.put(pkg, new LinkedHashSet<>());
159     for (Map.Entry<String, Map<String, Integer>> entry : weights.entrySet()) {
160         graph.get(entry.getKey()).addAll(entry.getValue().keySet());
161     }
162
163     // Iteratively break SCCs using weight-based rank, then assign levels
164     // via SCC-collapsed DAG longest-path. Remaining SCCs (equal-rank peers)
165     // are collapsed to the same level without cutting any edge.
166     Set<String> backEdgeKeys = new LinkedHashSet<>(); // "from\0to" -- tracked for R3
167     boolean changed = true;
168     while (changed) {
169         changed = false;
170         for (StronglyConnectedComponent scc : new TarjanSCCFinder(graph).findSCCs()) {
171             if (scc.getSize() < 2) continue;
172             Set<String> members = scc.getMembers();
173
174             // Compute weight-based rank for each member within this SCC
175             Map<String, Double> rank = new HashMap<>();
176             for (String m : members) {
177                 int out = 0, in = 0;
178                 for (String other : members) {
179                     if (other.equals(m)) continue;
180                     out += weights.getDefault(m, Map.of()).getOrDefault(other, 0);
181                     in += weights.getDefault(other, Map.of()).getOrDefault(m, 0);
182                 }
183                 rank.put(m, (out - in) / (double) Math.max(1, out + in));

```

```

184     }
185
186     // Identify back-edges: from→to where rank(from) < rank(to) - threshold
187     for (String from : new ArrayList<>(members)) {
188         for (String to : new ArrayList<>(graph.getOrDefault(from, Set.of()))) {
189             if (!members.contains(to)) continue;
190             if (rank.get(from) < rank.get(to) - RANK_THRESHOLD) {
191                 String key = from + "\0" + to;
192                 if (backEdgeKeys.add(key)) {
193                     graph.get(from).remove(to);
194                     changed = true;
195                 }
196             }
197         }
198     }
199 }
200
201
202 // Assign package levels: SCC-collapsed DAG → longest-path
203 List<StronglyConnectedComponent> sccs = new TarjanSCCFinder(graph).findSCCs();
204 Map<String, StronglyConnectedComponent> pkgToScc = new HashMap<>();
205 for (StronglyConnectedComponent scc : sccs) {
206     for (String m : scc.getMembers()) pkgToScc.put(m, scc);
207 }
208
209 // SCC dependency graph (between SCC IDs)
210 Map<Integer, Set<Integer>> sccDeps = new HashMap<>();
211 for (StronglyConnectedComponent scc : sccs) {
212     sccDeps.put(scc.getId(), new HashSet<>());
213     for (String m : scc.getMembers()) {
214         for (String to : graph.getOrDefault(m, Set.of())) {
215             StronglyConnectedComponent toScc = pkgToScc.get(to);
216             if (toScc != null && toScc.getId() != scc.getId()) {
217                 sccDeps.get(scc.getId()).add(toScc.getId());
218             }
219         }
220     }
221 }
222
223 // Longest-path levels on the SCC-collapsed DAG. The previous
224 // "childPkgUsedLevel" lift (which inflated a parent's level by
225 // the class levels its child packages used) is gone -- layout
226 // positioning lives on localLevel now, so architectureLevel
227 // can be a direct dependency-chain depth.
228 Map<Integer, Integer> sccLevels = new HashMap<>();
229 for (StronglyConnectedComponent scc : sccs) sccLevels.put(scc.getId(), 0);
230 boolean lvlChanged = true;
231 while (lvlChanged) {
232     lvlChanged = false;
233     for (StronglyConnectedComponent scc : sccs) {
234         int maxDep = -1;
235         for (int depId : sccDeps.getOrDefault(scc.getId(), Set.of())) {
236             maxDep = Math.max(maxDep, sccLevels.getOrDefault(depId, 0));
237         }
238         int newLevel = maxDep >= 0 ? maxDep + 1 : 0;
239         if (sccLevels.get(scc.getId()) != newLevel) {
240             sccLevels.put(scc.getId(), newLevel);
241             lvlChanged = true;
242         }
243     }
244 }
245

```

```

246 // Apply levels to all packages
247 Map<String, DomainModel.CalculatedElementInfo> pkgInfos = model.getAllPackages();
248 for (StronglyConnectedComponent scc : sccs) {
249     int level = sccLevels.get(scc.getId());
250     for (String member : scc.getMembers()) {
251         DomainModel.CalculatedElementInfo pkg = pkgInfos.get(member);
252         if (pkg != null) pkg.setArchitectureLevel(level);
253     }
254 }
255
256 // Store the weighted graph and identified back-edges in the model
257 model.setPackageEdgeWeights(weights);
258 model.setPackageBackEdges(backEdgeKeys);
259
260 // Populate package dependencies (unweighted) for reverse-dependency tracking
261 for (Map.Entry<String, Map<String, Integer>> entry : weights.entrySet()) {
262     DomainModel.CalculatedElementInfo pkg = pkgInfos.get(entry.getKey());
263     if (pkg != null) entry.getValue().keySet().forEach(pkg::addDependency);
264 }
265 }
266
267 /**
268  * Builds the weighted inter-package dependency graph.
269  * weight(P_A → P_B) = total method-call count from classes in P_A's subtree
270  * to classes in P_B. Method calls are a stronger signal than distinct-class
271  * counts: a class called hundreds of times clearly dominates one called once,
272  * making SCC-breaking direction unambiguous. Intra-subtree calls are excluded.
273  * Child-to-ancestor and parent-to-child edges are included.
274  * Each call count is propagated to all ancestor packages up to the point where
275  * the ancestor would enter the same subtree as the target.
276  */
277 private Map<String, Map<String, Integer>> buildWeightedPackageGraph(
278     DomainModel model, DependencyModel rawModel) {
279     Map<String, Map<String, Integer>> weights = new HashMap<>();
280     for (String pkg : model.getAllPackages().keySet()) weights.put(pkg, new LinkedHashMap<>());
281
282     Set<String> allPkgNames = weights.keySet();
283
284     for (DomainModel.CalculatedElementInfo cls : model.getAllClasses().values()) {
285         String leafPkg = extractPackageName(cls.fullName);
286         if (leafPkg == null || !allPkgNames.contains(leafPkg)) continue;
287
288         // Count method calls per target package using raw model data
289         Map<String, Integer> callCountPerPkg = new LinkedHashMap<>();
290         DependencyModel.ClassInfo rawCls = rawModel.getClass(cls.fullName);
291         if (rawCls != null) {
292             for (DependencyModel.MethodInfo method : rawCls.methods.values()) {
293                 for (Map.Entry<String, Integer> call : method.methodCalls.entrySet()) {
294                     // Method call key format: "ownerClass.methodName"
295                     // Find the target class by checking known dependencies
296                     String calledKey = call.getKey();
297                     for (String dep : cls.dependencies) {
298                         if (calledKey.startsWith(dep + ".")) {
299                             String toPkg = extractPackageName(dep);
300                             if (toPkg != null && !leafPkg.equals(toPkg)
301                                 && allPkgNames.contains(toPkg)) {
302                                 callCountPerPkg.merge(toPkg, call.getValue(), Integer::sum);
303                             }
304                             break;
305                         }
306                     }
307                 }
308             }
309         }
310     }
311 }

```

```

308     }
309 }
310 // Fall back to 1 per target package for structural deps with no call data
311 for (String dep : cls.dependencies) {
312     String toPkg = extractPackageName(dep);
313     if (toPkg != null && !leafPkg.equals(toPkg) && allPkgNames.contains(toPkg)) {
314         callCountPerPkg.putIfAbsent(toPkg, 1);
315     }
316 }
317
318 // Propagate each weight up to ancestor packages. Parent->child
319 // edges are no longer filtered here -- the rank-based SCC breaker
320 // decides direction based on actual call-count weight. Layout
321 // positioning is the LocalLevelCalculator's job (step 6), so the
322 // global package level can be a direct dependency-chain count.
323 for (Map.Entry<String, Integer> entry : callCountPerPkg.entrySet()) {
324     String toPkg = entry.getKey();
325     int callCount = entry.getValue();
326     String ancestor = leafPkg;
327     while (ancestor != null && allPkgNames.contains(ancestor)) {
328         if (ancestor.equals(toPkg)) break;
329         weights.get(ancestor).merge(toPkg, callCount, Integer::sum);
330         ancestor = extractPackageName(ancestor);
331     }
332 }
333 }
334 return weights;
335 }
336
337 // -----
338 // Step 5 -- reverse dependencies
339 // -----
340
341 private void updateDependentRelationships(DomainModel model) {
342     for (DomainModel.CalculatedElementInfo cls : model.getAllClasses().values()) {
343         for (String dep : cls.dependencies) {
344             DomainModel.CalculatedElementInfo d = model.getClass(dep);
345             if (d != null) d.addDependent(cls.fullName);
346         }
347     }
348     for (DomainModel.CalculatedElementInfo pkg : model.getAllPackages().values()) {
349         for (String dep : pkg.dependencies) {
350             DomainModel.CalculatedElementInfo d = model.getPackage(dep);
351             if (d != null) d.addDependent(pkg.fullName);
352         }
353     }
354 }
355
356 // -----
357 // Utilities
358 // -----
359
360 private static boolean isInSameSubtree(String a, String b) {
361     return a.startsWith(b + ".") || b.startsWith(a + ".");
362 }
363
364 private static String extractPackageName(String className) {
365     if (className == null || !className.contains(".")) return null;
366     return className.substring(0, className.lastIndexOf('.'));
367 }
368 }

```

The companion class LocalLevelCalculator implements Phase 3 (Step 6).

```
1  /*
2  * Copyright 2026 Weigend AM GmbH & Co.KG
3  *
4  * Licensed under the Apache License, Version 2.0 (the "License");
5  * you may not use this file except in compliance with the License.
6  * You may obtain a copy of the License at
7  *
8  *     http://www.apache.org/licenses/LICENSE-2.0
9  *
10 * Unless required by applicable law or agreed to in writing, software
11 * distributed under the License is distributed on an "AS IS" BASIS,
12 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
13 * See the License for the specific language governing permissions and
14 * limitations under the License.
15 */
16 package de.weigend.s202.domain.architecture;
17
18 import de.weigend.s202.domain.DomainModel;
19 import de.weigend.s202.domain.DomainModel.CalculatedElementInfo;
20 import de.weigend.s202.graph.StronglyConnectedComponent;
21 import de.weigend.s202.graph.TarjanSCCFinder;
22 import de.weigend.s202.reader.DependencyModel;
23
24 import java.util.ArrayList;
25 import java.util.HashMap;
26 import java.util.HashSet;
27 import java.util.LinkedHashSet;
28 import java.util.List;
29 import java.util.Map;
30 import java.util.Set;
31
32 /**
33  * Assigns a per-parent layer position ({@code localLevel}) to every
34  * element in a {@link DomainModel}. Each parent package is treated as a
35  * self-contained scope; only class dependencies whose source and
36  * target both live in the parent's subtree contribute to the sibling-only
37  * weighted graph. Refs that leave the parent are ignored -- they belong to
38  * the global {@code architectureLevel} via the package hierarchy.
39  *
40  * <p>Within each parent's sibling graph:
41  * <ol>
42  *   <li>Tarjan finds SCCs.</li>
43  *   <li>SCCs of size > 1 are broken one edge at a time. The default
44  *     strategy evaluates every candidate edge in the whole SCC and cuts
45  *     the edge whose removal decomposes the cyclic component the most.
46  *     Package-architecture contradictions, edge weight, and rank are used
47  *     only as tie-breakers.</li>
48  *   <li>Longest-path on the SCC-collapsed DAG assigns a {@code layer}
49  *     (= local layer index) per sibling. Layer 0 = bottom of the box,
50  *     higher values = visually higher.</li>
51  * </ol>
52  *
53  * <p>Single-child containers stay at layer 0 -- no graph, no work.
54  *
55  * <p>The algorithm intentionally mirrors the structure of
56  * {@link LevelCalculator}'s package-level computation, but applied
57  * locally and using a different graph. Unlike package-level computation,
58  * local SCC breaking is layout-oriented: it aims to stabilize the visible
59  * architecture by reducing cyclic local structure, not to minimize the
60  * number of visible upward edges. Local cycles are not surfaced as tangles;
61  * the user-visible tangle list remains the global one.
```



```

62 */
63 public class LocalLevelCalculator {
64
65     private static final double RANK_THRESHOLD = 0.1;
66     private final BreakMode breakMode;
67
68     public LocalLevelCalculator() {
69         this(BreakMode.MAX_CYCLE_BREAK);
70     }
71
72     LocalLevelCalculator(BreakMode breakMode) {
73         this.breakMode = breakMode;
74     }
75
76     enum BreakMode {
77         /**
78          * Legacy local heuristic: remove every edge whose source has lower
79          * weighted rank than its target by more than {@link #RANK_THRESHOLD}.
80          */
81         RANK,
82         /**
83          * Default local heuristic: inspect the whole SCC and remove one edge
84          * whose cut maximally decomposes the cyclic component.
85          */
86         MAX_CYCLE_BREAK
87     }
88
89     public void assign(DomainModel domain, DependencyModel rawModel) {
90         Map<String, List<CalculatedElementInfo>> childrenByParent = groupChildrenByParent(domain);
91         for (Map.Entry<String, List<CalculatedElementInfo>> entry : childrenByParent.entrySet()) {
92             assignForParent(entry.getValue(), domain, rawModel);
93         }
94     }
95
96     private void assignForParent(List<CalculatedElementInfo> siblings,
97                                 DomainModel domain, DependencyModel rawModel) {
98         if (siblings.size() <= 1) {
99             return; // single child stays at layer 0
100         }
101
102         Set<String> siblingFqns = new LinkedHashSet<>();
103         for (CalculatedElementInfo c : siblings) {
104             siblingFqns.add(c.fullName);
105         }
106
107         Map<String, Map<String, Integer>> weights = buildSiblingGraph(siblingFqns, domain, rawModel);
108         Map<String, Integer> layers = computeLayers(weights, siblings, siblingFqns);
109         for (CalculatedElementInfo s : siblings) {
110             s.setLocalLevel(layers.getOrDefault(s.fullName, 0));
111         }
112     }
113
114     /**
115      * Build the weighted sibling-only edge map. For every class C in any
116      * sibling's subtree, every dep into another sibling's subtree
117      * contributes {@code callCount(C, dep)} weight; deps that leave the
118      * parent's subtree or stay inside the same sibling are skipped.
119      */
120     private Map<String, Map<String, Integer>> buildSiblingGraph(Set<String> siblingFqns,
121                                                                 DomainModel domain,
122                                                                 DependencyModel rawModel) {
123         Map<String, Map<String, Integer>> weights = new HashMap<>();

```

```

124     for (String s : siblingFqns) {
125         weights.put(s, new HashMap<>());
126     }
127     for (CalculatedElementInfo cls : domain.getAllClasses().values()) {
128         String fromSibling = containingSibling(cls.fullName, siblingFqns);
129         if (fromSibling == null) {
130             continue;
131         }
132         for (String dep : cls.dependencies) {
133             if (domain.getClass(dep) == null) {
134                 continue; // external -- not in any sibling
135             }
136             String toSibling = containingSibling(dep, siblingFqns);
137             if (toSibling == null || toSibling.equals(fromSibling)) {
138                 continue; // out of parent OR intra-sibling
139             }
140             int w = callCount(cls.fullName, dep, rawModel);
141             weights.get(fromSibling).merge(toSibling, w, Integer::sum);
142         }
143     }
144     return weights;
145 }
146
147 /**
148  * Walk up the fqcn package chain until the first ancestor that is one
149  * of the siblings, or return {@code null} when none of the ancestors
150  * is -- the class lives outside the parent's subtree.
151  */
152 private static String containingSibling(String fqcn, Set<String> siblingFqns) {
153     String current = fqcn;
154     while (current != null && !current.isEmpty()) {
155         if (siblingFqns.contains(current)) {
156             return current;
157         }
158         int dot = current.lastIndexOf('.');
159         if (dot < 0) {
160             return null;
161         }
162         current = current.substring(0, dot);
163     }
164     return null;
165 }
166
167 private static Map<String, List<CalculatedElementInfo>> groupChildrenByParent(DomainModel domain) {
168     Map<String, List<CalculatedElementInfo>> result = new HashMap<>();
169     for (CalculatedElementInfo cls : domain.getAllClasses().values()) {
170         result.computeIfAbsent(parentOf(cls.fullName), k -> new ArrayList<>()).add(cls);
171     }
172     for (CalculatedElementInfo pkg : domain.getAllPackages().values()) {
173         result.computeIfAbsent(parentOf(pkg.fullName), k -> new ArrayList<>()).add(pkg);
174     }
175     return result;
176 }
177
178 private static String parentOf(String fqcn) {
179     int dot = fqcn.lastIndexOf('.');
180     return dot < 0 ? "" : fqcn.substring(0, dot);
181 }
182
183 /**
184  * Method-call count from class {@code from} to class {@code to}.
185  * Falls back to 1 for structural deps (extends/implements/field type)

```

```

186      * where no method-call info exists -- same convention the global
187      * weighted package graph uses.
188      */
189      private static int callCount(String from, String to, DependencyModel rawModel) {
190          DependencyModel.ClassInfo cls = rawModel.getClass(from);
191          if (cls == null) {
192              return 1;
193          }
194          int count = 0;
195          String prefix = to + ".";
196          for (DependencyModel.MethodInfo method : cls.methods.values()) {
197              for (Map.Entry<String, Integer> call : method.methodCalls.entrySet()) {
198                  if (call.getKey().startsWith(prefix)) {
199                      count += call.getValue();
200                  }
201              }
202          }
203          return count > 0 ? count : 1;
204      }
205
206      // ----- SCC-collapsed longest path with configurable SCC break -----
207
208      private Map<String, Integer> computeLayers(Map<String, Map<String, Integer>> weights,
209          List<CalculatedElementInfo> siblings,
210          Set<String> nodes) {
211          Map<String, Set<String>> graph = new HashMap<>();
212          for (String n : nodes) {
213              graph.put(n, new HashSet<>(weights.getOrDefault(n, Map.of()).keySet()));
214          }
215          Map<String, CalculatedElementInfo> siblingByName = new HashMap<>();
216          for (CalculatedElementInfo sibling : siblings) {
217              siblingByName.put(sibling.fullName, sibling);
218          }
219
220          // Iteratively break local SCCs, then assign layers on the remaining DAG.
221          // MAX_CYCLE_BREAK is the default; RANK remains available as a legacy
222          // comparison mode for tests and experiments.
223          boolean changed = true;
224          while (changed) {
225              changed = false;
226              for (StronglyConnectedComponent scc : new TarjanSCCFinder(graph).findSCCs()) {
227                  if (scc.getSize() < 2) {
228                      continue;
229                  }
230                  Set<String> members = scc.getMembers();
231                  Map<String, Double> rank = new HashMap<>();
232                  for (String m : members) {
233                      int out = 0;
234                      int in = 0;
235                      for (String other : members) {
236                          if (other.equals(m)) {
237                              continue;
238                          }
239                          out += weights.getOrDefault(m, Map.of()).getOrDefault(other, 0);
240                          in += weights.getOrDefault(other, Map.of()).getOrDefault(m, 0);
241                      }
242                      rank.put(m, (out - in) / (double) Math.max(1, out + in));
243                  }
244                  if (breakMode == BreakMode.MAX_CYCLE_BREAK) {
245                      EdgeCutCandidate best = bestCycleBreakingEdge(graph, weights, members, rank, siblingByName);
246                      if (best != null) {
247                          graph.get(best.from()).remove(best.to());

```

```

248         changed = true;
249     }
250     continue;
251 }
252 // Legacy path: remove all rank-based back-edges found in this SCC.
253 for (String from : new ArrayList<>(members)) {
254     for (String to : new ArrayList<>(graph.getDefault(from, Set.of()))) {
255         if (!members.contains(to)) {
256             continue;
257         }
258         if (rank.get(from) < rank.get(to) - RANK_THRESHOLD) {
259             graph.get(from).remove(to);
260             changed = true;
261         }
262     }
263 }
264 }
265 }
266
267 // SCC-collapsed longest-path level assignment.
268 List<StronglyConnectedComponent> sccs = new TarjanSCCFinder(graph).findSCCs();
269 Map<String, StronglyConnectedComponent> nodeToScc = new HashMap<>();
270 for (StronglyConnectedComponent scc : sccs) {
271     for (String m : scc.getMembers()) {
272         nodeToScc.put(m, scc);
273     }
274 }
275 Map<Integer, Set<Integer>> sccDeps = new HashMap<>();
276 for (StronglyConnectedComponent scc : sccs) {
277     sccDeps.put(scc.getId(), new HashSet<>());
278     for (String m : scc.getMembers()) {
279         for (String to : graph.getDefault(m, Set.of())) {
280             StronglyConnectedComponent toScc = nodeToScc.get(to);
281             if (toScc != null && toScc.getId() != scc.getId()) {
282                 sccDeps.get(scc.getId()).add(toScc.getId());
283             }
284         }
285     }
286 }
287 Map<Integer, Integer> sccLayers = new HashMap<>();
288 for (StronglyConnectedComponent scc : sccs) {
289     sccLayers.put(scc.getId(), 0);
290 }
291 boolean lvlChanged = true;
292 while (lvlChanged) {
293     lvlChanged = false;
294     for (StronglyConnectedComponent scc : sccs) {
295         int maxDep = -1;
296         for (int depId : sccDeps.get(scc.getId())) {
297             maxDep = Math.max(maxDep, sccLayers.get(depId));
298         }
299         int newLayer = maxDep >= 0 ? maxDep + 1 : 0;
300         if (sccLayers.get(scc.getId()) != newLayer) {
301             sccLayers.put(scc.getId(), newLayer);
302             lvlChanged = true;
303         }
304     }
305 }
306
307 Map<String, Integer> result = new HashMap<>();
308 for (StronglyConnectedComponent scc : sccs) {
309     int layer = sccLayers.get(scc.getId());

```

```

310         for (String m : scc.getMembers()) {
311             result.put(m, layer);
312         }
313     }
314     return result;
315 }
316
317 private static EdgeCutCandidate bestCycleBreakingEdge(Map<String, Set<String>> graph,
318                                                       Map<String, Map<String, Integer>> weights,
319                                                       Set<String> members,
320                                                       Map<String, Double> rank,
321                                                       Map<String, CalculatedElementInfo> siblingByName) {
322     EdgeCutCandidate best = null;
323     for (String from : members) {
324         for (String to : graph.getDefault(from, Set.of())) {
325             if (!members.contains(to)) {
326                 continue;
327             }
328             EdgeCutCandidate candidate = new EdgeCutCandidate(
329                 from,
330                 to,
331                 cycleBreakScore(graph, members, from, to),
332                 architectureContradiction(from, to, siblingByName),
333                 weights.getDefault(from, Map.of()).getDefault(to, 0),
334                 rank.getDefault(to, 0.0) - rank.getDefault(from, 0.0));
335             if (best == null || candidate.compareTo(best) > 0) {
336                 best = candidate;
337             }
338         }
339     }
340     return best;
341 }
342
343 /**
344  * Score how much one candidate cut decomposes the current SCC. The score
345  * compares the original SCC size squared with the sum of squared SCC sizes
346  * after removing the candidate edge. A cut that splits one large cycle into
347  * several smaller components therefore wins over a cut that leaves the SCC
348  * almost intact.
349  */
350 private static int cycleBreakScore(Map<String, Set<String>> graph,
351                                   Set<String> members,
352                                   String cutFrom,
353                                   String cutTo) {
354     Map<String, Set<String>> copy = new HashMap<>();
355     for (String member : members) {
356         copy.put(member, new HashSet<>());
357         for (String to : graph.getDefault(member, Set.of())) {
358             if (members.contains(to) && !(member.equals(cutFrom) && to.equals(cutTo))) {
359                 copy.get(member).add(to);
360             }
361         }
362     }
363
364     int after = 0;
365     for (StronglyConnectedComponent scc : new TarjanSCCFinder(copy).findSCCs()) {
366         after += scc.getSize() * scc.getSize();
367     }
368     return members.size() * members.size() - after;
369 }
370
371 /**

```

```

372     * Tie-breaker for package siblings: if an edge runs against the already
373     * computed package architectureLevel, prefer cutting it over an otherwise
374     * equivalent package edge. Class edges return 0 because classes are placed
375     * inside the package frame, not used to redefine that frame.
376     */
377     private static int architectureContradiction(String from,
378                                                  String to,
379                                                  Map<String, CalculatedElementInfo> siblingByName) {
380         CalculatedElementInfo fromInfo = siblingByName.get(from);
381         CalculatedElementInfo toInfo = siblingByName.get(to);
382         if (fromInfo == null || toInfo == null) {
383             return 0;
384         }
385         if (!"PACKAGE".equals(fromInfo.type) || !"PACKAGE".equals(toInfo.type)) {
386             return 0;
387         }
388         return fromInfo.architectureLevel < toInfo.architectureLevel ? 1 : 0;
389     }
390
391     private record EdgeCutCandidate(String from,
392                                    String to,
393                                    int cycleBreakScore,
394                                    int architectureContradiction,
395                                    int weight,
396                                    double rankBackEdgeScore)
397         implements Comparable<EdgeCutCandidate> {
398         /**
399          * Higher is better. Order of preference:
400          * 1. always cut an edge that contradicts the package architecture
401          *    hypothesis first -- Phase 3 must never reorder packages relative
402          *    to what Phase 2 computed,
403          * 2. among architecture-equivalent candidates, decompose the local SCC
404          *    as much as possible,
405          * 3. cut the weaker edge,
406          * 4. fall back to the legacy rank signal.
407          */
408         @Override
409         public int compareTo(EdgeCutCandidate other) {
410             int cmp = Integer.compare(architectureContradiction, other.architectureContradiction);
411             if (cmp != 0) {
412                 return cmp;
413             }
414             cmp = Integer.compare(cycleBreakScore, other.cycleBreakScore);
415             if (cmp != 0) {
416                 return cmp;
417             }
418             cmp = Integer.compare(other.weight, weight);
419             if (cmp != 0) {
420                 return cmp;
421             }
422             return Double.compare(rankBackEdgeScore, other.rankBackEdgeScore);
423         }
424     }
425 }

```